

What Nobody Tells You After the Demo Works



AI Coach John



Follow

 Repost

Building the model is the **easy** part

Almost every AI tutorial ends where real engineering begins. Getting a RAG pipeline or an agent to work once, on your machine, for one user, is a weekend project. Turning that into a product that serves thousands of users, at predictable cost, with acceptable latency and a UX people trust, that is a completely different discipline.

THE 12 AREAS WE ARE COVERING

- Cost optimization
- Latency budgets
- Streaming
- Feedback loops
- Multitenancy
- Observability
- Token economics
- UX for uncertainty
- Human in the loop
- Personalization
- Quotas
- Prod readiness



Works once



Works for 10k users



AI Coach John



PROFITBRIDGE
Strive For Better Future

Follow

 Repost

The Unit Economics of AI

Know your cost per request before you know anything else. Unlike traditional software, marginal cost scales with usage.

```
def cost_per_req(in_tok, out_tok):  
    in_cost = (in_tok / 1M) * 3.00  
    out_cost = (out_tok / 1M) * 15.00  
    return in_cost + out_cost
```

```
cost = cost_per_req(4000, 400)
```

```
# ~$0.018 per request
```

```
# At 50,000 requests/month:
```

```
# ~$900/month
```



Output tokens usually cost several times more than input tokens. A verbose system prompt is cheap, but a model that rambles is expensive.



AI Coach John



PROFITBRIDGE
Strive For Better Future

Follow

↻ Repost

The Layered Approach

Cut cost in order of effort, not in order of excitement. The cheapest fixes are usually the most effective.

1

Shorten prompts

Trim redundant instructions; free, immediate

2

Cap output length

Set max_tokens deliberately

3

Prompt caching

Reuse a static system prompt/context

+

Semantic caching, Routing, Batch

More effort, but significant savings at scale



AI Coach John



Follow

↻ Repost

Not every request deserves your **best model**

Classifies incoming requests by complexity and routes them to the appropriately-sized model. Often the single biggest cost lever.

Simple



Small fast model (\$)

FAQ, classification, extraction

Moderate



Mid tier model (\$\$)

Standard RAG answers

Complex



Frontier model (\$\$\$)

Nuanced analysis, multi-step



The classifier itself must be cheap.



AI Coach John



PROITBRIDGE
Strive For Better Future

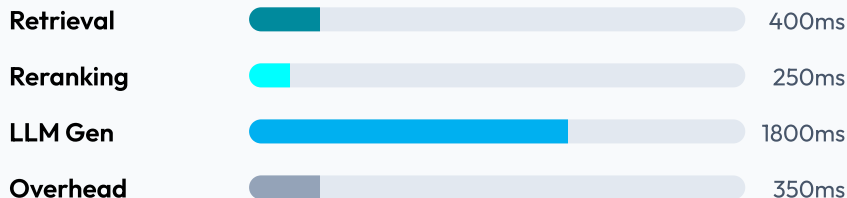
Follow

↻ Repost

Spend it deliberately

Treat total response time as a fixed budget that each pipeline stage draws from.

TARGET: 3000MS



 **Generation dominates. Optimize output length or stream.**



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost

Latency: Perceived vs Actual

Streaming changes the experience without changing the speed.

⌚ ACTUAL

Total time until the full response is complete.

👁️ PERCEIVED

Time until the user sees something happening.

```
for chunk in llm.stream(prompt):  
    yield chunk.content  
# user sees text appearing immediately
```

Streaming the first token in 400ms feels dramatically faster than waiting 2,800ms for a complete response.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

UX for Uncertainty

Traditional software is deterministic. AI products aren't. Your UX has to account for that honestly.

” **Show sources:** let users verify claims themselves

⚠ **Express confidence:** flag low-confidence answers

✎ **Make correction easy:** obvious path to "edit this"

👋 **Never fake certainty:** admit when info is missing

Mock UI



↺ Retry

✎ Edit



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↺↻ Repost



Where to Put the Human

Not every step needs approval. Design HITL by consequence, not by uniform policy.

Low stakes

Auto-execute

Read-only, reversible (answering a Q)

Medium

Execute + review after

Logged & correctable (drafting content)

High stakes

Approve BEFORE

Irreversible actions (sending emails/money)

```
app = graph.compile(  
    checkpointer=checkpointer,  
    interrupt_before=["send_email", "pay"]  
)
```



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

Loops That Actually Improve the Product

Collecting thumbs up or down is the easy part. Using it is the real work. Feedback that goes into a database and never comes back out is theater.

IMPLICIT

- Did they copy?
- Did they regenerate?
- Session abandoned?

EXPLICIT

- ✓ Thumbs up/down
- ✓ Star ratings
- ✓ **User edits (gold!)**

Route it somewhere:

- Negative feedback → new test cases
- Corrections → few-shot examples
- Failure patterns → prompt improvements



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 **Repost**

Feedback Loop Contamination

Careful: your feedback loop can poison itself. If you use your own model's outputs to generate eval data, the model's mistakes become "ground truth."

Model output → "Ground Truth" → Trains next model

⚠ Errors amplify each cycle

🛡 GUARDRAILS

- ✓ Keep **human-verified golden set** that is never generated by the model.
- ✓ Spot-check LLM-generated eval data against human judgment regularly.
- ✓ Watch out for metrics improving while user complaints stay the same (drift).



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost



Personalization Without Rebuilding the Model

Three layers, from cheapest to most involved.

1. Prompt

Inject preferences (tone, role) in system prompt.
(Cheapest, instant)

2. Retrieval

Filter vector store by user. Retrieve only their docs. (Multitenancy)

3. Memory

Persist facts across sessions and inject relevant ones. (Most involved)

```
# Retrieval-level: filter by user
results = vector_store.search(
    query,
    filter={"user_id": current_user.id}
)
```





Keeping Users' Data **Separate**

The mistake here isn't a bug, it's a breach. Three isolation strategies:



Metadata filtering: One shared index, filtered at query time.

Cheapest, but one missing filter leaks data.



Namespace per tenant: Separate namespace in same DB.

Stronger isolation, balanced cost.



Separate infrastructure: Dedicated DB per tenant.

Safest, priciest. Best for enterprise.



Enforce filters at a shared data layer, never at individual call sites.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

Rate Limits & Quotas

Two different things. Handle them differently in your UX.

RATE LIMITS

- Protects your system
- Per minute/second
- "Slow down, try again"
- Resets quickly

QUOTAS

- Protects your business
- Per billing period
- "Upgrade to continue"
- Business boundary

```
if rate_limiter.exceeded(user, per_minute=20):  
    raise RateLimitError("Too fast.")  
  
if user.tokens >= user.plan.quota:  
    raise QuotaExceededError("Upgrade.")
```



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost

When things go wrong

AI products fail in ways traditional software doesn't. Provider outages, rate limits, timeouts. Design the fallback explicitly.



Fallback chain: try secondary model automatically



Cached response: stale answer > error page



Reduced functionality: keyword search > full fail



Honest error states: explain what is happening



Primary → Fallback → Cheap → Cached

(User always gets something useful)



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost

What to **Actually** Log

You can't debug an AI product with print statements.

PER REQUEST TRACE

- **Inputs/outputs:** prompt sent, response received (manage PII)
- **Token counts & cost:** attribute to user/tenant/feature
- **Latency breakdown:** per pipeline stage, not just total
- **Retrieval trace:** which chunks retrieved & scores
- **Model version:** essential for diagnosing quality changes
- **Feedback signal:** linked back to the specific request



Knowing which feature and tenant drives spend lets you optimize the right thing.



AI Coach John



Follow

↻ Repost

A Continuous Practice

Shipping isn't the end of evaluation. It is the start.



Pre-ship: Regression suite on every change, fails the build if score drops.



Post-ship: Sample live traffic and score automatically. Data drifts.



Alert on drops: Treat falling quality like a rising error rate (an incident).



Close the loop: Prod failures become new test cases.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost

Production Readiness Checklist

- ✓ Cost per request measured & forecast
- ✓ Latency budget defined & measured per stage
- ✓ Model fallback chain configured
- ✓ Rate limits & quotas enforced
- ✓ Multitenancy isolated at data layer
- ✓ Human-in-the-loop on irreversible actions
- ✓ Feedback wired into evaluation set
- ✓ Full request tracing with cost attribution
- ✓ Regression evaluation running in CI
- ✓ Graceful degradation path implemented
- ✓ Honest UX: citations, confidence, easy correction



Complete this before you call it shipped.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

How AI products actually die in production

Cross-tenant data leak

One missing filter; the most damaging failure.

Unbounded costs

No quotas, one power user burns the budget.

Silent quality decay

No post-ship eval, nobody notices until users churn.

Over-automation

No human gate on irreversible actions.

Latency creep & No fallback

Adding stages makes it sluggish; outages kill it.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 **Repost**

The model is **20%** of the product



Cost, latency, UX, feedback, isolation, and graceful failure are what turn a working prototype into something people pay for and keep using. The demo proves it is possible. This layer is what makes it a product.

 **Save this as your AI product engineering checklist. Follow for more.**



AI Coach John



Follow

 **Repost**